

RESEARCH ARTICLE

Explaining Customer Churn Through Unified Perturbation-Based XAI Soft Computing

SIMONA-VASILICA OPREA^{ID} AND ADELA BĂRA^{ID}

Department of Economic Informatics and Cybernetics, Bucharest University of Economic Studies, 010374 Bucharest, Romania

Corresponding author: Simona-Vasilica Oprea (simona.oprea@csie.ase.ro)

This work was supported in part by the Ministry of Research, Innovation and Digitization, CNCS/CCCDI—UEFISCDI within PNCDI IV, under Project COFUND-CETP-SMART-LEM-1; and in part by the Clean Energy Transition Partnership (CETP) through the 2022 CETP Joint Call for Research Proposals co-funded by European Commission (<https://cetpartnership.eu/funding-agencies-and-call-modules>) under Grant GAN 101069750.

ABSTRACT In this research, we propose a methodology for explaining AI models and determining the critical features for retaining customers to prevent churn. By aggregating feature importance values across multiple explainers, our research harmonizes different explainers into a unified importance score, reducing bias and variability. It focuses on impactful features by filtering rows with positive recall drops and comparing results with established techniques like SHAP and LIME. Customers on month-to-month contracts and those with high monthly charges are at the highest churn risk, particularly when tenure is below 20 months. The results also indicate strong alignment between feature importance rankings derived from SHAP and the proposed multiple Perturbation-Based Explainers (PBE). Spearman Rank Correlation, Kendall Tau and Jaccard Similarity, demonstrate high agreement between the methods considering several classifiers. Furthermore, Kolmogorov-Smirnov and Wasserstein distance tests confirm minimal differences in their feature importance distributions. A comparative evaluation of SHAP, LIME, PBE based on fidelity, consistency and runtime shows that SHAP offers the highest fidelity but is computationally expensive (289s). LIME achieves moderate fidelity with faster runtime (35s), while PBE is the most efficient (18s), making it ideal for real-time or resource-constrained scenarios. Runtime estimates for SHAP reveal significant scalability challenges. A sensitivity analysis using alternative objectives (e.g., F1-score drop) confirms the stability of feature rankings, demonstrating the robustness of the proposed metric-agnostic framework. Additionally, scalability and deployment considerations are discussed, highlighting the suitability of the approach for integration into real-time CRM systems and drift-aware operational environments.

INDEX TERMS Churn prediction, explainable artificial intelligence, perturbation-based explainer, runtime, scalability.

I. INTRODUCTION

A. GENERAL CONCEPT AND MOTIVATION

As Artificial Intelligence (AI) models powering applications in healthcare, finance, energy, retailing [1], autonomous vehicle and beyond become increasingly sophisticated, their decision-making processes often turn opaque. This lack of transparency makes it challenging to understand how AI systems reach their conclusions, highlighting the importance of

explaining AI models [2]. The need to explain AI algorithms arises from multiple considerations. Trust and transparency are foundational to the successful deployment and adoption of AI systems, particularly in regulated or sensitive domains such as healthcare and finance [3]. By making AI decisions interpretable, stakeholders better understand and trust the outcomes, reducing the skepticism towards AI. Moreover, explainability is important to ensure ethical and fair decision-making [4]. Further, AI systems trained on biased data may unintentionally propagate societal inequities, but interpretability allows researchers to identify and address

The associate editor coordinating the review of this manuscript and approving it for publication was Zhengmao Li^{ID}.

these biases [5]. Accountability is another crucial factor, particularly in areas like credit scoring, where decisions may have major consequences. Explainable AI (XAI) enables organizations to justify their actions, fostering a sense of responsibility [6]. However, providing these explanations is not without challenges. Complex models, such as deep neural networks, often trade off interpretability for accuracy [7], [8]. Scaling explanations for big data or complex models is computationally intensive [9].

The development of explainability methods addresses these challenges. Techniques like SHAP (SHapley Additive exPlanations) and LIME (Local Interpretable Model-agnostic Explanations) offer insights into model behavior across algorithm types, while visualization tools help elucidate the impact of specific features [10], [11]. Perturbation-based approaches further complement these efforts by exploring how changes in input features influence model predictions [12], [13]. While SHAP and LIME are widely used methods for explaining AI models, they have several shortcomings that impact their reliability, scalability and applicability.

B. SHORTCOMINGS OF SHAP AND LIME

One of the main challenges of SHAP is its computational intensity. Calculating Shapley values involves evaluating all possible feature subsets, which grows exponentially with the number of features. While approximate algorithms such as TreeSHAP have been developed for specific model types like tree-based ensembles, SHAP remains computationally expensive for big data or deep learning [14], [15]. Runtime is a critical evaluation metric in engineering applications requiring fast computations like finance, optimization, cybersecurity and autonomous driving, where rapid processing is essential [16]. Similarly, LIME requires training multiple surrogate linear models for each prediction, making it resource-intensive when applied to big data. Both SHAP and LIME rely on perturbing the input data to assess feature importance. LIME generates synthetic data around a given input to train its local surrogate model, while SHAP uses sampling methods to estimate marginal contributions of features. These perturbations may lead to unrealistic data points that do not reflect the actual distribution of the dataset, potentially resulting in misleading explanations. This issue is particularly problematic for high-dimensional or unstructured data, where perturbations may not preserve meaningful relationships.

C. THE ROLE OF XAI IN KNOWLEDGE-BASED ECONOMY

Retaining existing customers is often more cost-effective than acquiring new ones, as loyal customers provide consistent revenue streams, valuable referrals and increased opportunities for cross-selling and upselling [17], [18]. Thus, churn prediction is an essential tool in reducing the rate at which customers stop doing business with a company. This process involves analyzing customer data to identify those at risk of leaving. Once high-risk customers are identified, targeted interventions can be implemented to retain them.

XAI has become a key component in churn prediction and customer retention, addressing the need for transparency and interpretability in Machine Learning (ML) models [19], [20], [21] as they often function without clear reasoning. XAI techniques like SHAP and LIME make it possible to interpret the influence of specific features on predictions [22]. This interpretability builds trust among stakeholders and helps them make informed decisions based on model outputs. However, the runtime of these techniques is significantly higher, and it represents a major drawback. For businesses, understanding timely why customers churn is just as important as identifying who is at risk, as it allows for the development of more effective retention strategies, including personalized strategies [23].

To address shortcomings of SHAP and LIME, we propose a multiple PBE approach aiming to offer an alternative method for explaining AI models, significantly reducing the runtime and identifying key features to enhance customer retention and reduce churn. By aggregating feature importance values across multiple explainers, our approach unifies their insights into a single importance score. Incorporating diverse explainers and robust feature ranking minimizes the bias or variability of individual methods, resulting in a more rapid and comprehensive evaluation of feature importance.

The main contribution of our work lies in the integration of three components: (i) a recall-driven perturbation framework tailored to churn prediction, prioritizing the identification of high-risk customers; (ii) a structured aggregation strategy that combines four complementary perturbation mechanisms (RNP, FA, IM, AP) into a unified importance score to reduce method-specific bias; and (iii) an evaluation pipeline that assesses ranking agreement, distribution similarity, fidelity, consistency and runtime. While perturbation-based explainers exist in prior literature, they typically rely on a single intervention mechanism or do not aggregate multiple sensitivity perspectives into a unified score. Our framework differentiates itself by formalizing perturbation diversity into a bias-reduced, recall-aligned importance metric validated through statistical similarity analysis against SHAP and LIME.

II. LITERATURE REVIEW

The integration of XAI into online gaming was explored [24], addressing the limitations of traditional AI models that lack transparency. The authors proposed a GXAI workflow that leverages multiview data sources and transparent black-box models. They developed classifiers and explainers for various gaming perspectives. Experiments using real-world datasets demonstrate the effectiveness of the approach in game cheating detection and player churn prediction, providing rational explanations. Additionally, [25] addressed player churn in online games by proposing a data-driven decision support framework that utilized XAI to predict churn and analyze its causes. By identifying potential reasons for churn, the framework guided game publishers in making targeted interventions. Another research focused on modeling user

engagement in online recreational games through an unsupervised learning framework [26]. Engagement was treated as a continuous temporal process, analyzed using principal component analysis to derive key metrics. The geometric variability of user trajectories emerged as a strong predictor, with higher variability indicating sustained engagement. Other authors introduced EMGCD [27], an explainable multi-view game cheating detection framework leveraging XAI. By combining cheating classifiers with explainers across multiple views, the framework provided individual, local and global explanations. The results showed that it is adaptable to related tasks, including explainable recommender systems and churn prediction.

A churn prediction methodology for the telecom industry to better identify customers likely to churn was proposed [28]. Seven ML models were trained on 3 public datasets, achieving superior performance in recall, F1-score and AUC. The research also explored new feature crosses that proved impactful in churn prediction. The best models were explained using SHAP to enhance interpretability. Another research highlighted the application of Layer-wise Relevance Propagation (LRP) for explainability in deep learning [29]. Applied to credit card fraud detection and telecom customer churn prediction, LRP demonstrated superior effectiveness over LIME and SHAP for local and holistic explanations while being significantly faster (1)-2s vs. 22-108s).

In a computational experiment on bank customer churn data, [30] demonstrated the explanatory and predictive capabilities of monotonic decision rules. The data, partially ordinal in nature, exhibited monotonic relationships between certain attributes and churn outcomes. The Variable Consistency Dominance-based Rough Set Approach (VC-DRSA) was used to structure the data before inducing monotonic decision rules. Supervised learning was conducted with an extended VC-DRSA implementation in RuLeStudio and RuleVisualization programs, enabling parameterized rule experimentation and detailed visualization. The monotonic decision rules provided insights into customer behavior, aligning with explainable AI principles for transparent decision-making. The research compared the predictive performance of these rules with well-known ML models. Another research utilized an interpretable ML model to predict customer churn transparently, leveraging the Berka database from a Czech bank for feature extraction via deep clustering, along with a Kaggle dataset [31]. To address dataset imbalance, Synthetic Minority Over Sampling Techniques (SMOTE) were applied before training, validating and testing 4 tree-based and four standard ML models. Random Forest (RF) achieved high performance, with 99% accuracy, 98.5% recall and 98.5% F1-score on the Berka dataset and 85% accuracy, 77.5% recall and 77% F1-score on the Kaggle dataset. LIME and SHAP ensured model accountability. Additionally, [32] analyzed churn using a synthetic Kaggle dataset processed with SMOTETomek to balance classes and improve separability. Two advanced ML models,

RF and Light Gradient-Boosting Machine (LGBM) were applied and evaluated using metrics like precision, sensitivity, F1 score, accuracy and Brier score. Probability calibration techniques enhanced reliability, and SHAP explained feature importance, offering insights for personalized retention strategies.

Research on AI models for customer churn prediction proved extensive in B2C but insufficient in B2B environments. Reference [33] addressed this gap by introducing a novel RFID (Recency, Frequency, Importance, Duration) model based on customer interactions with contact centers. Focusing on a business software distribution company, it aimed to design a methodological process to analyze the XAI for customer churn in a B2B context. Moreover, [34] presented CENTAURO, an AI-based approach for predicting customer loyalty to support effective loyalty programs and decision-making. It extended the RFM (Recency, Frequency, Monetary) model to track these values over time and across product categories. Experiments using a benchmark dataset evaluated the extended RFM model with classification algorithms like Logistic Regression (LR), RF and eXtreme Gradient Boost (XGB). SHAP was applied to explain how RFM values influenced the loyalty profiles predicted by the model.

Further, [35] proposed an XAI model to understand customer decisions regarding non-life insurance coverage. Using Shapley values from an accurate XGB algorithm, the method applied similarity clustering to explain customer behavior. Embedded in an Insurtech platform, it provided real-time insights into factors influencing customer purchases and churn. Empirical analysis on insurance micro-policy data validated the model, revealing effective grouping of customers by shared characteristics to predict buying and churn behavior accurately. Another research addressed the high dropout rates in online learning by analyzing students' daily learning data to predict dropouts [36]. A ML model was trained using students' basic attributes, learning progress and conversations with teachers. The model achieved 96.4% accuracy and correctly predicted 78% of actual dropouts. SHAP values were used to interpret feature importance, enhancing the model's transparency and applicability in online education. Also, [37] used SHAP in education to explain the academic problems.

Previous studies emphasized churn prediction and behavior analysis in gaming, telecom, banking, business software distribution, insurance and education using XAI methods like SHAP, LIME and others. These domains highlight the broad applicability of AI and XAI techniques in addressing churn, engagement and decision-making challenges. Also, previous studies focused on improving SHAP [38], [39] by introducing model-specific approximations for deep learning or tree-based models and stochastic estimators that sample permutations or feature subsets [40], [41], [42]. These improvements aim to optimize computational efficiency, either by approximating Shapley values more

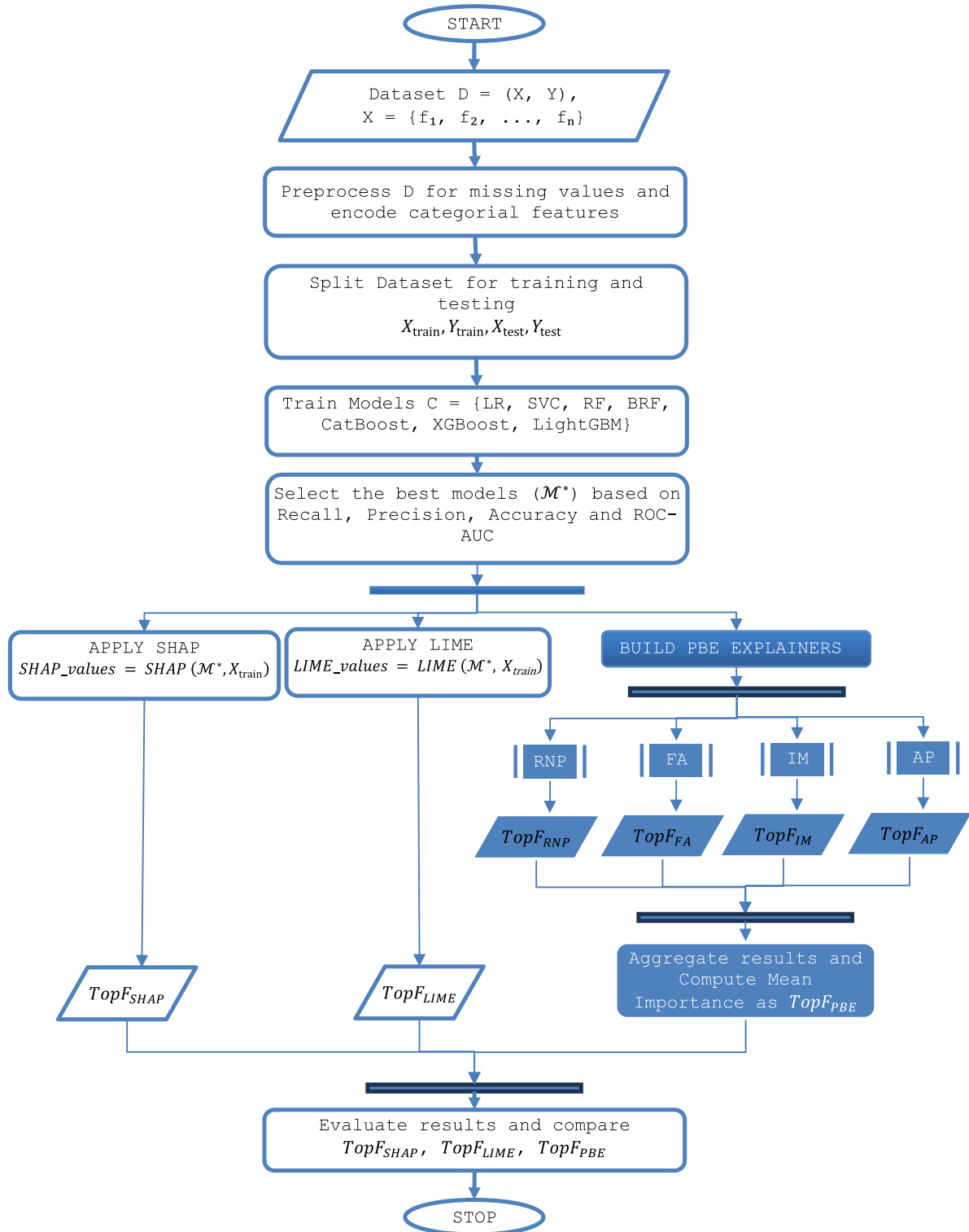


FIGURE 1. Flowchart of the proposed methodology for building multiple PBE.

effectively, removing less critical features or amortizing SHAP values to streamline calculations [14]. Beyond refining SHAP, researchers have developed alternative XAI techniques that offer enhanced performance [43], [44], [45], [46].

III. RESEARCH METHODOLOGY

The flowchart of the proposed methodology is depicted in Figure 1.

In this paper, we propose an alternative for explaining AI algorithms and obtaining the most important features

to retain customers, providing effective strategies to avoid churn. By aggregating the importance values for each feature, our approach harmonizes the different explainers into a unified importance score. Considering multiple explainers and robust feature ranking, our approach reduces the bias or variability of any single method, leading to a more comprehensive assessment of feature importance.

The proposed methodology aggregates and ranks feature importance values across multiple explainers (Random Noise Perturbation - RNP, Feature Ablation - FA, Input Marginalization - IM and Adversarial Perturbation - AP) for a better understanding of feature importance. The features' importance values from different explainers are concatenated and rows with positive recall drop are filtered to focus on impactful features. The results of PBE are compared with the SHAP and LIME features importance.

The four perturbation-based explainers were selected to capture complementary sensitivity dimensions rather than redundant effects: (a) RNP measures stochastic robustness under random variability, (b) FA quantifies structural dependency through retraining, (c) IM evaluates global contribution under distribution-neutral replacement, and (d) AP captures worst-case local sensitivity. While all are perturbation-based, they differ in intervention mechanism (noise injection, removal, marginal substitution, adversarial shift) supporting their complementarity and therefore probe distinct aspects of model reliance. The aggregation step therefore reduces method-specific bias while preserving diverse sensitivity signals.

A. PRE-PROCESSING AND CLASSIFIERS

Unnecessary columns for classification (such as customerID) are dropped. Others (tenure column) are converted to a float, and blank values (in the TotalCharges column) are replaced with NaN before being converted to float, ensuring that these columns can be used for numerical computations. Categorical columns are processed using *LabelEncoder*, which transforms non-numeric data into numerical labels. For interpretability, for each column, the encoder generates and prints the mapping between original categories and their numerical equivalents. The target variable (Churn) is separated into Y , while the remaining features are assigned to X . The dataset is split into training and testing subsets using *train_test_split*, with 30% of the data reserved for testing. A classifier is then initialized and trained on the training data, handling potential imbalances in the target variable. Predictions are made on the test dataset using the trained classifier. Performance is evaluated using several metrics, including *accuracy* (the proportion of correct predictions), *recall* (the ability to identify positive cases), *ROC-AUC* (the model's ability to discriminate between positive and negative classes) and *precision* (the proportion of correct positive predictions).

Accuracy

$$= \frac{TP + TN}{Total\ Predictions(TP + TN + FP + FN)} \quad (1)$$

Recall(Sensitivity or True Positive Rate)

$$= \frac{TP}{TP + FN} \quad (2)$$

Precision(Positive Predictive Value)

$$= \frac{TP}{TP + FP} \quad (3)$$

$$AUC = \int_0^1 TPR(FPR) dFPR \quad (4)$$

Where *FPR* - False Positive Rate; *TP* - True Positives, correctly predicted positive cases; *TN* - True Negatives, correctly predicted negative cases; *FP* - False Positives, incorrectly predicted positive cases; *FN* - False Negatives, incorrectly predicted negative cases; *AUC* - Area Under the Curve.

Recall is important when dealing with customer churn because it measures the ability of a model to correctly identify all customers likely to churn. Failing to identify potential churners (false negatives) results in missed opportunities for intervention and retention. A low recall means that the model is not capturing a significant portion of actual churners, which could lead to higher customer losses. In churn prediction, recall ensures that most at-risk customers are flagged, allowing businesses to focus their retention efforts, such as personalized offers or enhanced customer support, on those who need it most. By identifying as many potential churners as possible, the business can reduce the risk of losing revenue due to customer attrition, as retaining existing customers is generally more cost-effective than acquiring new ones. While precision, which measures the proportion of correctly predicted churners, is also important to avoid wasting resources on non-churners, businesses often prioritize recall because the cost of failing to identify a churner is higher than targeting a non-churner. Maximizing recall ensures that the business can take proactive measures to retain a greater number of customers and minimize churn-related revenue losses. Thus, recall is the key metric in customer churn analysis.

Seven classifiers are considered to predict the telecommunication customers' churn: LR, Support Vector Classifier (SVC), RF, Balanced RF (BRF), Cat Boost (CB), XGB and LGMB. The difference between RF classifier with the *class_weight* option and BRF lies in how they handle class imbalance during training. Both aim to address imbalanced datasets where one class significantly outnumbers the other, but they do so in different ways. The *class_weight* parameter modifies the cost of misclassifying a sample based on its class. It adjusts the weight of each class when calculating the impurity of splits in the decision trees. The weight for a class c is denoted as:

$$w_c = \frac{Total\ samples}{Number\ of\ classes \times Samples\ in\ class\ c} \quad (5)$$

This weight is applied in the Gini impurity or entropy calculation when splitting nodes. For a node with K classes, the

weighted Gini impurity G_w is computed as:

$$G_w = \sum_{k=1}^K w_k \times p_k (1 - p_k) \quad (6)$$

where w_k - weight for class k (from `class_weight`) and p_k - proportion of samples in class k at the node.

The weights influence the splitting criterion, encouraging the algorithm to prioritize splitting on features that reduce impurity for underrepresented classes.

BRF uses class weights and also ensures balance at the sampling level. During the training of each tree, it under-samples the majority class to match the size of the minority class. This ensures that each tree sees a balanced subset of the data. For a dataset with two classes: majority class size ($N_{majority}$) and minority class size ($N_{minority}$), at each tree, the classifier samples $N_{minority}$ examples from both the majority and minority classes. This balance affects the probability p_k in the Gini impurity calculation:

$$G_{balanced} = \sum_{k=1}^K p_k (1 - p_k) \quad (7)$$

where $p_k = \frac{\text{Samples of class } k}{\text{Total samples in the undersampled dataset}}$.

Since the class sizes are equal in each tree, p_k is unaffected by class imbalance, resulting in a balanced evaluation of splits. Thus, `class_weight` may handle imbalance effectively in many cases, but it struggles when the imbalance is extreme because it relies solely on weighting. BRF explicitly balances the dataset for each tree, which often results in better performance.

B. DESCRIPTION OF SHAP AND LIME

Both SHAP and LIME are widely used tools for explaining ML predictions, with SHAP focusing on global consistency and fairness and LIME prioritizing local interpretability. SHAP Python package uses game theory's Shapley values to fairly allocate the contribution of each feature to the model's prediction. The Shapley value for a feature i is given by:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)] \quad (8)$$

where F - the set of all features; S is a subset of features excluding i ; $f(S)$ - the model prediction with features in S present.

In SHAP, interventional algorithms compute Shapley values by defining the value function for a set of attributes at a query point as the expected model prediction under an interventional distribution. This ensures that the influence of features is properly isolated, avoiding spurious correlations from the data. The value function for a subset of features S at a query point x is defined as the expected model output when features in S are fixed to their values in x , and other features are sampled from their conditional distribution:

$$v(S, x) = \mathbb{E}[f(x_S, X_{\sim S})] \quad (9)$$

where $f(x)$ - the model's prediction function; x_S - the values of features in the subset S for the query point;

$X_{\sim S}$ - features not in S , sampled from the interventional distribution.

The interventional distribution assumes that features not in S are independent of features in S when conditioned on their respective observed values. This is achieved by sampling these features from their marginal or conditional distributions.

$$P(X_{\sim S} | X_S = x_S) \quad (10)$$

The Shapley value for a feature i at a query point x is computed as:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} [v(S \cup \{i\}, x) - v(S, x)] \quad (11)$$

where F - the set of all features; S - a subset of features excluding i ; $v(S, x)$ - the expected model output under the interventional distribution.

In Kernel SHAP, the value function $v(S, x)$ is estimated by sampling data points and weighting them according to the interventional distribution:

$$v(S, x) = \frac{\sum_{j=1}^M w_j \cdot f(x_S, X_{\sim S}^{(j)})}{\sum_{j=1}^M w_j} \quad (12)$$

where $X_{\sim S}^{(j)}$ - samples from the interventional distribution; w_j - weights determined by the Kernel SHAP algorithm.

For tree-based models, the interventional algorithm leverages the tree structure to efficiently compute Shapley values. The value function $v(S, x)$ is computed by averaging the outputs of all leaf nodes reachable based on S , with other features sampled from the interventional distribution.

$$v(S, x) = \sum_{l \in \text{leaves}} \text{weight}_l \cdot \text{mean}_l(S, x) \quad (13)$$

where weight_l - fraction of data points reaching leaf l ; $\text{mean}_l(S, x)$ - mean prediction in leaf l conditioned on the values in S .

SHAP computes feature interaction values under the interventional distribution:

$$\begin{aligned} \phi_{i,j} = \sum_{S \subseteq F \setminus \{i,j\}} \frac{|S|! (|F| - |S| - 2)!}{|F|!} [v(S \\ \cup \{i,j\}, x) - v(S \cup \{i\}, x) \\ - v(S \cup \{j\}, x) + v(S, x)] \end{aligned} \quad (14)$$

Eq. (14) quantifies the combined effect of features i and j . For the baseline or expected value ϕ_0 , the interventional distribution is used to compute the average model prediction. It is the baseline against which feature contributions (ϕ_i) are measured:

$$\phi_0 = E[f(X)] \quad (15)$$

LIME approximates the behavior of a complex model locally (around a specific instance). It generates a simpler interpretable model (e.g., linear regression) for a neighborhood of the instance to provide insight into the predictions. It solves the following optimization problem:

$$\text{Explanation}(x) = \underset{g \in G}{\operatorname{argmin}} \mathcal{L}(f, g, \pi_x) + \Omega(g) \quad (16)$$

where f - the complex model; g - the interpretable model (e.g., linear); $\mathcal{L}$ is the loss function measuring the fidelity of g to f in the local neighborhood π_x ; $\Omega(g)$ is a regularization term to enforce simplicity in g .

C. MULTIPLE PBE

The primary goal of the noise-based feature importance using a perturbation approach such as *Random Noise Perturbation (RNP)* is to evaluate how adding random noise to individual features affects the model's performance, helping identify the most critical features for accurate predictions. For each feature f_i in the test set X_{test} , a random noise is added to the selected feature, scaled based on the feature's range (calculated as the difference between the maximum and the minimum values), and the perturbed values are clipped to ensure they remain within the feature's original range. For each feature, the *range* is calculated as:

$$\text{range}_i = \max(X_{\text{test}}[f_i]) - \min(X_{\text{test}}[f_i]) \quad (17)$$

It represents the spread of values the feature can take. A fixed noise level (e.g., set $\text{noise_level} = 0.1$) is defined, representing the percentage of the feature range used to generate noise. For example, with a noise level of 0.1, the noise added will be within 10% of the feature's range. Noise values are generated randomly using a uniform distribution as in eq.(17):

$$\text{noise}_i \sim U(-\text{noise_level} \times \text{range}_i, \text{noise_level} \times \text{range}_i) \quad (18)$$

Thus, it is ensured that the noise is both positive and negative, distributed symmetrically around zero. After adding noise, the values are clipped to remain within the original range of the feature. This ensures that the perturbed values do not exceed the natural limits of the features, preserving the validity of the data. Based on the modified values of the features, the algorithm creates a perturbed copy of the test set ($X_{\text{perturbed}}$) as in eq. (18):

$$\begin{aligned} X_{\text{perturbed}}[f_i] \\ = \text{clip}(X_{\text{test}}[f_i] + \text{noise}_i, \min(X_{\text{test}}[f_i]), \max(X_{\text{test}}[f_i])) \end{aligned} \quad (19)$$

The perturbation process introduces controlled randomness to the feature values, mimicking real-world variability or measurement errors. This allows the model's sensitivity to the changes in individual features to be evaluated, highlighting the impact of each feature on the model's predictions. For each perturbed sub-set, the model makes predictions, and

the resulting performance metrics (Accuracy, Recall, ROC-AUC and Precision) are computed. The drop in performance (Recall Drop) is calculated by subtracting the perturbed metric from the baseline metric. The difference represents the impact of perturbing the feature on the model's performance. The results are stored and sorted by Recall Drop to emphasize the features most critical for capturing positive cases (Churn). The runtime of the model ($\text{runTime}_{\text{RNP}}$) is also calculated. Features with larger performance drops are deemed more important for the model's predictions, while features with smaller drops are either less influential or more robust to noise. This perturbation provides a quantitative way to understand feature importance based on sensitivity to noise. Random noise and "zeroing" features were also implemented in [47].

Feature Ablation (FA) iteratively removes individual features or groups of features from the input and measures the change in the model's output. For each feature f_i , new input sets for training and testing are obtained as in eq. (19):

$$\begin{aligned} X_{\text{train}}^{-f_i} &\leftarrow X_{\text{train}} \setminus f_i \\ X_{\text{test}}^{-f_i} &\leftarrow X_{\text{test}} \setminus f_i \end{aligned} \quad (20)$$

Then, the model is retrained on these modified sets and a feature importance analysis is performed. It involves systematically removing one feature at a time from the dataset and observing the impact on the recall to assess the importance of each feature. Previous studies also implemented explaining by removing features or permutations [41], [48].

Input Marginalization (IM) replaces individual feature values with their marginal distribution (e.g., mean or median) and observes the prediction change. For each feature f_i , the values are replaced with the mean values:

$$X_{\text{marginalized}}[f_i] = \text{mean}(X_{\text{test}}[f_i]) \quad (21)$$

The model predicts the target using the marginalized values of the input ($X_{\text{marginalized}}$). The approach assesses feature importance by evaluating the impact on the recall when each feature in the test dataset is replaced with its mean or median value. This approach measures how sensitive the model is to each feature, identifying the most critical features.

Adversarial Perturbation (AP) creates adversarial examples by adding minimal perturbations designed to fool the model. It systematically introduces small adversarial perturbations to individual features in the test dataset and measures the impact on the recall. The resulting recall drops quantify each feature's sensitivity and importance to the model's predictions. A small perturbation factor is defined (e.g., set $\text{perturbation_factor} = 0.01$), which determines the magnitude of the adversarial perturbation applied to each feature. The perturbation is calculated in eq. (21) as a fraction of the range of the feature's values:

$$\begin{aligned} \text{perturbation}_i \\ = \text{perturbation_factor} \times (\max(X_{\text{test}}[f_i]) - \min(X_{\text{test}}[f_i])) \end{aligned} \quad (22)$$

This perturbation is then added to each feature f_i in the test dataset. To prevent the perturbed values from exceeding the feature's original range, the perturbed feature values are clipped using to ensure they remain within the minimum and maximum values observed in the test dataset:

$$X_{adversarial}[f_i] = \text{clip}(X_{test}[f_i] + \text{perturbation}_i, \min(X_{test}[f_i]), \max(X_{test}[f_i])) \quad (23)$$

The difference between the original performance metrics (computed on the unperturbed test dataset) and the perturbed metrics represents the performance drop (Recall Drop) caused by the perturbation. These performance drops indicate how critical the feature is to the model's recall.

The algorithm's steps for building multiple PBE are depicted in Table 1.

D. METRICS FOR EVALUATING THE RESULTS

Correlation analysis is conducted using metrics like *Spearman Rank Correlation* to evaluate the monotonic relationship between their rankings. *Ranking agreement* is evaluated using *Kendall's Tau*, which measures the agreement in feature ranking between SHAP and PBE.

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)} \quad (24)$$

$$\tau = \frac{(n_c - n_d)}{\frac{1}{2}n(n - 1)} \quad (25)$$

where d_i is the difference in ranks of feature i ; n is the number of features; n_c and n_d are the number of concordant and discordant pairs, respectively.

Additionally, *Top-K Agreement* compares the overlap in the top-k most important features identified by XAI methods.

$$\text{Top K Agreement} = \frac{|TopK_{PBE} \cap TopK_{SHAP} \cap TopK_{LIME}|}{K} \quad (26)$$

Feature importance disparity is assessed using *Mean Absolute Error* (MAE), *Root Mean Square Error* (RMSE) to evaluate the magnitude of differences and Jaccard similarity between feature importance vectors of SHAP and PBE values.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (\text{Imp}_{PBE}(f_i) - \text{Imp}_{other}(f_i))^2} \quad (27)$$

$$MAE = \frac{1}{n} \sum_{i=1}^n |\text{Imp}_{PBE}(f_i) - \text{Imp}_{other}(f_i)| \quad (28)$$

$$\text{Jaccard} = \frac{|TopK_{PBE} \cap TopK_{other}|}{|TopK_{PBE} \cup TopK_{other}|} \quad (29)$$

Distribution comparison is performed using the *Kolmogorov-Smirnov Test* to determine if the distributions of SHAP and PBE values are statistically similar. *Earth Mover's Distance* (*Wasserstein Distance*) is also used to measure the "distance" between the two distributions.

$$D = |F_{PBE}(x) - F_{other}(x)| \quad (30)$$

where $F_{PBE}(x)$ and $F_{other}(x)$ are cumulative distributions of importance scores.

$$EMD(P, Q) = \int |x - y| d\gamma(x, y) \quad (31)$$

where $\Gamma(P, Q)$ is the set of all possible distributions that transform P into Q .

Fidelity evaluates the agreement of predictions using perturbed features from $TopF_{PBE}$, $TopF_{SHAP}$ and $TopF_{LIME}$ with the original model predictions.

Runtime comparison measure ($runTime_{XAI}$) compares the execution time of each method.

Consistency evaluates the alignment between explanations for similar instances, with higher cosine similarity values between feature importance vectors indicating greater consistency.

IV. RESULTS

A. DESCRIPTION OF THE DATA FOR PREDICTING BEHAVIOR TO RETAIN CUSTOMERS

The dataset^{1,2} consists of 7,043 instances and 21 columns. It was downloaded from Kaggle platform and originally attributed to the IBM Sample Data Sets. Each row represents a telecommunication customer, each column contains customer's attributes such as: customers who left within the last month-the column is called Churn; services that each customer has signed up for: phone, multiple lines, internet, online security, online backup, device protection, tech support and streaming TV and movies; customer account information: tenure-how long they have been a customer, contract, payment method, paperless billing, monthly charges and total charges; some demographic info about customers: if they have partners and dependents. The data indicates a total of 7,043 observations in the dataset, with 5,174 customers labeled as non-churners (Churn=0) and 1,869 customers labeled as churners (Churn=1). The majority of the customers, approximately 73.5%, have not churned. This distribution shows that the dataset is imbalanced, with a significantly larger number of non-churners compared to churners. This imbalance imposed using several options such as *class_weight* for RF and *scale_pos_weight* for CB, XGB and LGBM.

B. CLASSIFIERS RESULTS

The results of the classification process are provided in Figures 2-5 and in Table 2. BRF, CB, XGB and LGBM provide the best results in terms of recall (>0.7), and they are selected as the best models for this dataset.

The lower part of the confusion matrices is more significant because it represents the recall, which indicates the model's ability to correctly identify the highest number of churners. The BRF outperforms the RF in terms of the total number of

¹<https://www.kaggle.com/code/kaanboke/catboost-lightgbm-xgboost-explained-by-shap/input>

²<https://community.ibm.com/community/user/businessanalytics/blogs/steven-macko/2019/07/11/telco-customer-churn-1113>

TABLE 1. Algorithm for integrating multiple PBE.

Input: Dataset $D = (X, Y)$, where $X = \{f_1, f_2, \dots, f_n\}$ and Y is the target variable
 Output: Ranked feature importance for RNP, FA, IM, AP, and PBE

STEP 1: PREPARE THE TRAIN AND TEST SETS. TRAIN CLASSIFICATION MODELS

Preprocess the dataset D for missing values and encoding categorical features

Split Dataset for training and testing

$(X_{\text{train}}, Y_{\text{train}}, X_{\text{test}}, Y_{\text{test}}) \leftarrow \text{TrainTestSplit}(X, Y, \text{test_size} = 0.2)$

Train Models:

Let $C = \{\text{LR}, \text{SVC}, \text{RF}, \text{BRF}, \text{CatBoost}, \text{XGBoost}, \text{LightGBM}\}$

For each $c \in C$:

Train c on $(X_{\text{train}}, Y_{\text{train}})$

Compute metrics $\mathcal{M}(c) = \{\text{Recall}_c, \text{Precision}_c, \text{Accuracy}_c, \text{ROC} - \text{AUC}_c\}$ on $(X_{\text{test}}, Y_{\text{test}})$

Select the best models $(\mathcal{M}^*)$ based on the metrics

$\mathcal{M}^* = \arg \max_{c \in C} (\text{Average}(\text{Recall}_c, \text{Precision}_c, \text{Accuracy}_c, \text{ROC} - \text{AUC}_c))$

Store Baseline Metrics:

$\text{Metrics}_{\text{baseline}} = \{\text{Recall}_{\text{base}}, \text{Precision}_{\text{base}}, \text{Accuracy}_{\text{base}}, \text{ROC-AUC}_{\text{base}}\}$

STEP 2: APPLY SHAP ON THE BEST MODELS

$\text{SHAP_values} = \text{SHAP}(\mathcal{M}^*, X_{\text{train}})$

Sort SHAP_values in descending order

$\text{TopF}_{\text{SHAP}} \leftarrow \text{SORT}(\text{SHAP_values}, \text{descending})$

Calculate runtime: $\text{runTime}_{\text{SHAP}}$

STEP 3: APPLY LIME ON THE BEST MODELS

$\text{LIME_values} = \text{LIME}(\mathcal{M}^*, X_{\text{train}})$

Sort LIME_values in descending order

$\text{TopF}_{\text{LIME}} \leftarrow \text{SORT}(\text{LIME_values}, \text{descending})$

Calculate runtime: $\text{runTime}_{\text{LIME}}$

STEP 4: BUILD AND APPLY RANDOM NOISE PERTURBATION (RNP)

Initialize:

$\text{TopF}_{\text{RNP}} \leftarrow \{\}, \text{noise_level} = 0.1$

Loop through features:

For each feature $f_i \in \{f_1, f_2, \dots, f_n\}$:

Compute range for f_i :

$\text{range}_i = \max(X_{\text{test}}[f_i]) - \min(X_{\text{test}}[f_i])$

Generate noise:

$\text{noise}_i \sim \mathcal{U}(-\text{noise_level} \times \text{range}_i, \text{noise_level} \times \text{range}_i)$

Perturb feature:

$X_{\text{perturbed}}[f_i] = \text{clip}(X_{\text{test}}[f_i] + \text{noise}_i, \min(X_{\text{test}}[f_i]), \max(X_{\text{test}}[f_i]))$

Predict the target using perturbed features:

$\hat{Y}_{\text{perturbed}} = \mathcal{M}^*(X_{\text{perturbed}})$

Compute Recall Drop for f_i :

$\text{Recall Drop}_i = \text{Recall}_{\text{base}} - \text{Recall}(Y_{\text{test}}, \hat{Y}_{\text{perturbed}})$

Store Recall:

$\text{TopF}_{\text{RNP}}[f_i] = \text{Recall Drop}_i$

Sort Results:

$\text{TopF}_{\text{RNP}} \leftarrow \text{SORT}(\text{TopF}_{\text{RNP}}, \text{Recall Drop}, \text{descending})$

Calculate runtime: $\text{runTime}_{\text{RNP}}$

STEP 5: BUILD AND APPLY FEATURE ABLATION (FA)

Initialize:

$\text{TopF}_{\text{FA}} \leftarrow \{\}$

Loop through features:

For each feature $f_i \in \{f_1, f_2, \dots, f_n\}$:

Remove feature f_i from the train and test sets:

$X_{\text{train}}^{-f_i} \leftarrow X_{\text{train}} \setminus \{f_i\}, X_{\text{test}}^{-f_i} \leftarrow X_{\text{test}} \setminus \{f_i\}$

Retrain model without f_i :

$\mathcal{M}_{-f_i}^* \leftarrow \text{Train}(X_{\text{train}}^{-f_i}, Y_{\text{train}})$

Predict the target:

$\widehat{Y}_{-f_i} = \mathcal{M}_{-f_i}^*(X_{\text{test}}^{-f_i})$

Compute Recall Drop:

$\text{Recall Drop}_i = \text{Recall}_{\text{base}} - \text{Recall}(Y_{\text{test}}, \widehat{Y}_{-f_i})$

Store Recall:

$\text{TopF}_{\text{FA}}[f_i] = \text{Recall Drop}_i$

TABLE 1. (Continued.) Algorithm for integrating multiple PBE.

Sort Results:
 $TopF_{FA} \leftarrow SORT(TopF_{FA}, \text{Recall Drop, descending})$
 Calculate runtime: $runTime_{FA}$
STEP 6: BUILD AND APPLY INPUT MARGINALIZATION (IM)
 Initialize:
 $TopF_{IM} \leftarrow \{\}$
 Loop through features:
 For each feature $f_i \in \{f_1, f_2, \dots, f_n\}$:
 Replace feature f_i with its mean values:
 $X_{marginalized}[f_i] = \text{mean}(X_{\text{test}}[f_i])$
 Predict target using $X_{marginalized}$:
 $\hat{Y}_{marginalized} = \mathcal{M}^*(X_{marginalized})$
 Compute Recall Drop:
 $\text{Recall Drop}_i = \text{Recall}_{\text{base}} - \text{Recall}(Y_{\text{test}}, \hat{Y}_{marginalized})$
 Store Recall:
 $TopF_{IM}[f_i] = \text{Recall Drop}_i$
 Sort Results:
 $TopF_{IM} \leftarrow SORT(TopF_{IM}, \text{Recall Drop, descending})$
 Calculate runtime: $runTime_{IM}$
STEP 7: BUILD AND APPLY ADVERSARIAL PERTURBATION (AP)
 Initialize:
 $TopF_{AP} \leftarrow \{\}$, $\text{perturbation_factor} = 0.01$
 Loop through features:
 For each feature $f_i \in \{f_1, f_2, \dots, f_n\}$:
 Compute perturbation for feature f_i :
 $\text{perturbation}_i = \text{perturbation_factor} \times (\max(X_{\text{test}}[f_i]) - \min(X_{\text{test}}[f_i]))$
 Perturb feature f_i :
 $X_{\text{adversarial}}[f_i] = \text{clip}(X_{\text{test}}[f_i] + \text{perturbation}_i, \min(X_{\text{test}}[f_i]), \max(X_{\text{test}}[f_i]))$
 Predict target with $X_{\text{adversarial}}[f_i]$:
 $\hat{Y}_{\text{adversarial}} = \mathcal{M}^*(X_{\text{adversarial}})$
 Compute Recall Drop:
 $\text{Recall Drop}_i = \text{Recall}_{\text{base}} - \text{Recall}(Y_{\text{test}}, \hat{Y}_{\text{adversarial}})$
 Store Recall:
 $TopF_{AP}[f_i] = \text{Recall Drop}_i$
 Sort Results:
 $TopF_{AP} \leftarrow SORT(TopF_{AP}, \text{Recall Drop, descending})$
 Calculate runtime: $runTime_{AP}$
STEP 8: BUILD MULTIPLE PERTURBATION-BASED EXPLAINERS (PBE)
 Aggregate importance and combine results from $TopF_{RNP}, TopF_{FA}, TopF_{IM}, TopF_{AP}$
 Compute Mean Importance as $PBE(f_i)$:
 For each feature $f_i \in \{f_1, f_2, \dots, f_n\}$:

$$TopF_{PBE}[f_i] = \frac{TopF_{RNP}[f_i] + TopF_{FA}[f_i] + TopF_{IM}[f_i] + TopF_{AP}[f_i]}{4}$$

 Sort Results:
 $TopF_{PBE} \leftarrow SORT(TopF_{PBE}, \text{Recall Drop, descending})$
STEP 9: EVALUATE RESULTS
 Compare $TopF_{PBE}$, $TopF_{SHAP}$, and $TopF_{LIME}$:
 - Correlation analysis using Spearman and Kendall correlations.
 - Ranking using Top-K agreement
 - Feature importance disparity using RMSE, MAE, Jaccard Similarity, Top-5 feature overlap.
 - Distribution comparison using Kolmogorov-Smirnov and Earth Mover's Distance tests, Fidelity, Runtime and Consistency metrics.
 - Visualize feature importance rankings for comparison.
 Compare the runtime of the models

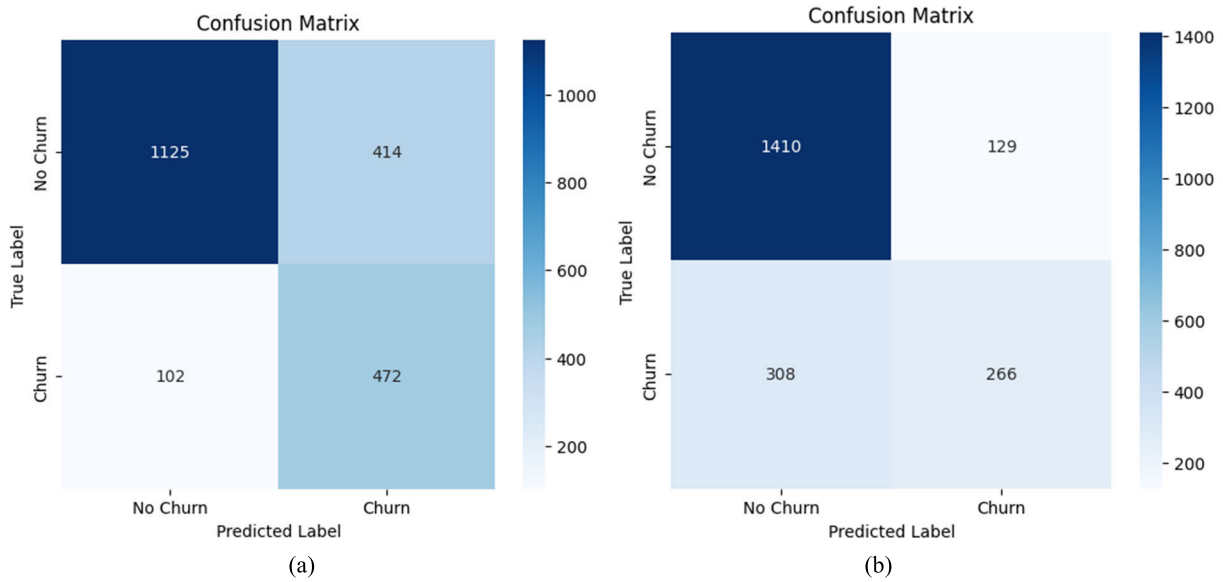


FIGURE 2. Confusion matrix with (a) BRF and (b) RF.

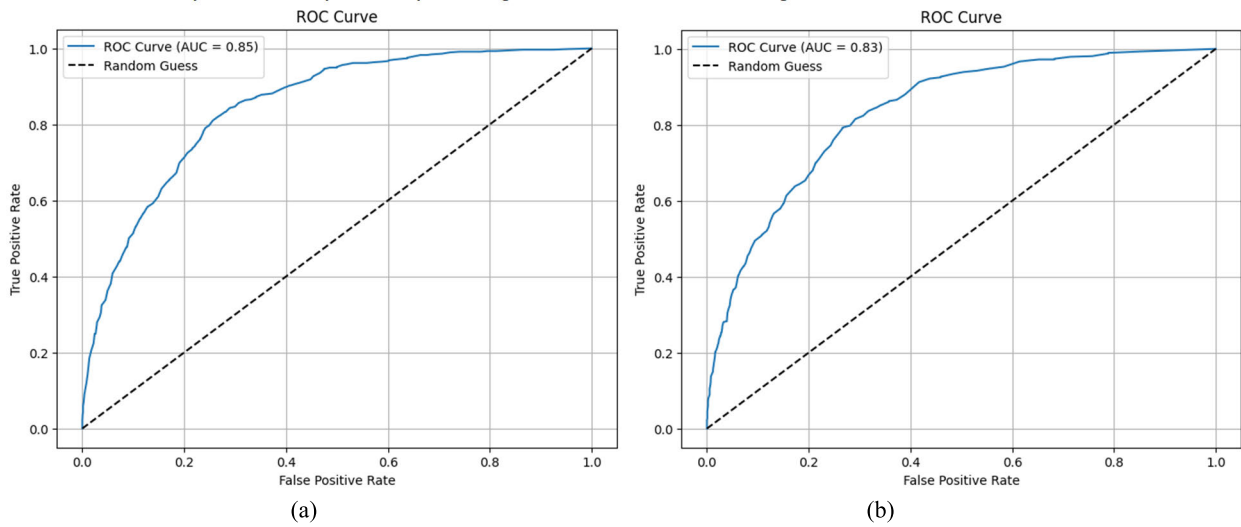


FIGURE 3. ROC-AUC with BRF and RF.

churners correctly predicted by the model. The highest number of churners was captured with CB model as in Figure 4.

C. SHAP AND LIME RESULTS

SHAP (TreeExplainer) is applied for the best models (BRF, CB, XGB, LGBM). and the impact on the model output magnitude is shown in Figures 6-7. From the recall, precision and F1 score point of view, the best model is BRF. Higher MonthlyCharges values are associated with lower

BRF+ SHAP values, meaning higher charges reduce the likelihood of the positive outcome (e.g., staying) as in Figure 8. For low charges, SHAP values are more positive, indicating affordability increases retention likelihood. Customers with Contract two-year tend to have higher SHAP values, suggesting they are less influenced by higher charges. Customers with Contract month-to-month are more negatively impacted by higher charges. MonthlyCharges is negatively correlated with the outcome, but the impact is modulated by the contract

$$Consistency = \frac{1}{n(n-1)} \sum_{i=1}^n \sum_{j \neq i} cosine_similarity(F_{XAI}(i), F_{XAI}(j)) \quad (32)$$

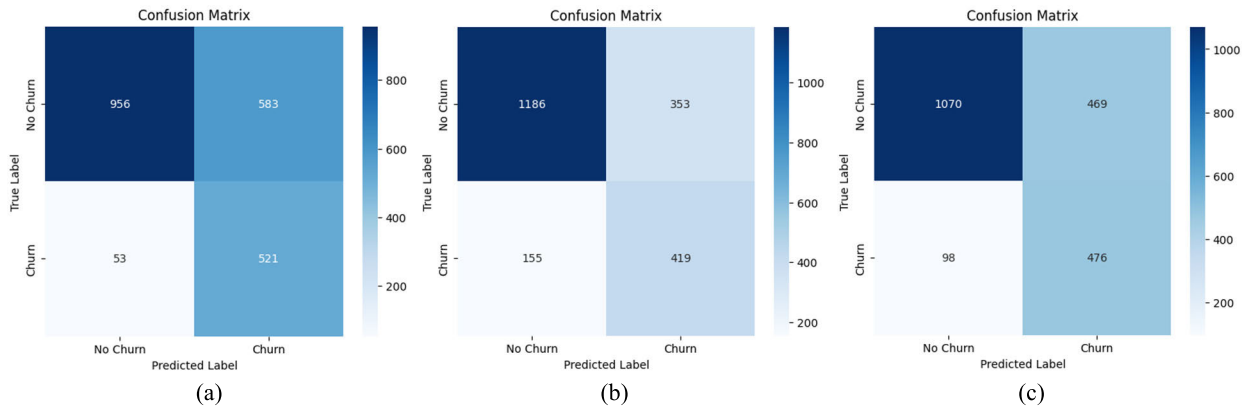


FIGURE 4. Confusion matrix with (a) CB, (b) XGB, (c) LGBM.

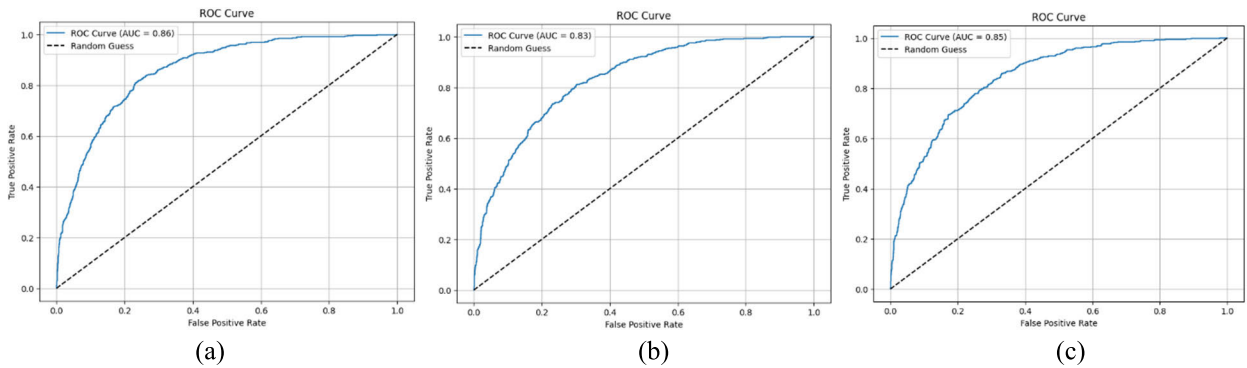


FIGURE 5. ROC-AUC with (a) CB, (b) XGB, (c) LGBM.

TABLE 2. Classification metrics for LR, SVC, RF, BRF, CB, XGB, LGBM.

Model	Accuracy	Recall	Precision
LR	0.7120	0.4112	0.6210
SVC	0.7419	0.5369	0.6319
RF	0.7932	0.5634	0.6734
BRF	0.7558	0.8223	0.6327
CB	0.6990	0.9077	0.4719
XGB	0.7596	0.7300	0.5427
LGBM	0.7317	0.8293	0.5037

type, with longer contracts buffering the negative effect of high charges. Monthly charges below approximately \$50-\$60 generally have positive or near-neutral SHAP values, indicating customers are less likely to churn. Monthly charges between \$60-\$90 have a mix of SHAP values, depending on factors like contract type or tenure. Customers in this range may churn if other risk factors are present. Monthly charges exceeding \$90-\$100 strongly correlate with negative SHAP values, indicating a high likelihood of churn, especially for customers on month-to-month contracts.

Customers with month-to-month contracts generally show lower SHAP values across all tenures. Even at higher tenure levels, the churn risk remains higher compared to cus-

tomers on longer contracts. These customers are consistently more likely to churn unless tenure is very high (above ~60 months), where SHAP values stabilize. Customers on one-year contracts show moderate SHAP values. Their likelihood of staying improves as tenure increases, but they may still churn if tenure is low. Customers with two-year contracts consistently show the highest SHAP values, even for short tenures. This indicates that two-year contracts strongly reduce churn risk, regardless of tenure. Customers with tenure below 20 months are at the highest risk of churn, especially if they are on month-to-month contracts. For tenure above 40 months, churn risk decreases significantly, particularly for customers on one-year or two-year contracts. Similar results in terms of features' importance are obtained with BRF + LIME as in Figure 9.

D. COMPARISON WITH MULTIPLE PBE

The results in Table 3 indicate a high relationship between SHAP values and multiple PBE values. The Spearman Rank Correlation of 0.8125 for BRF + SHAP and BRF + PBE suggests that SHAP and PBE have a highly positive monotonic relationship in terms of feature rankings. This indicates a good level of agreement on the ordering of features, although there are notable differences. Similarly, Kendall's

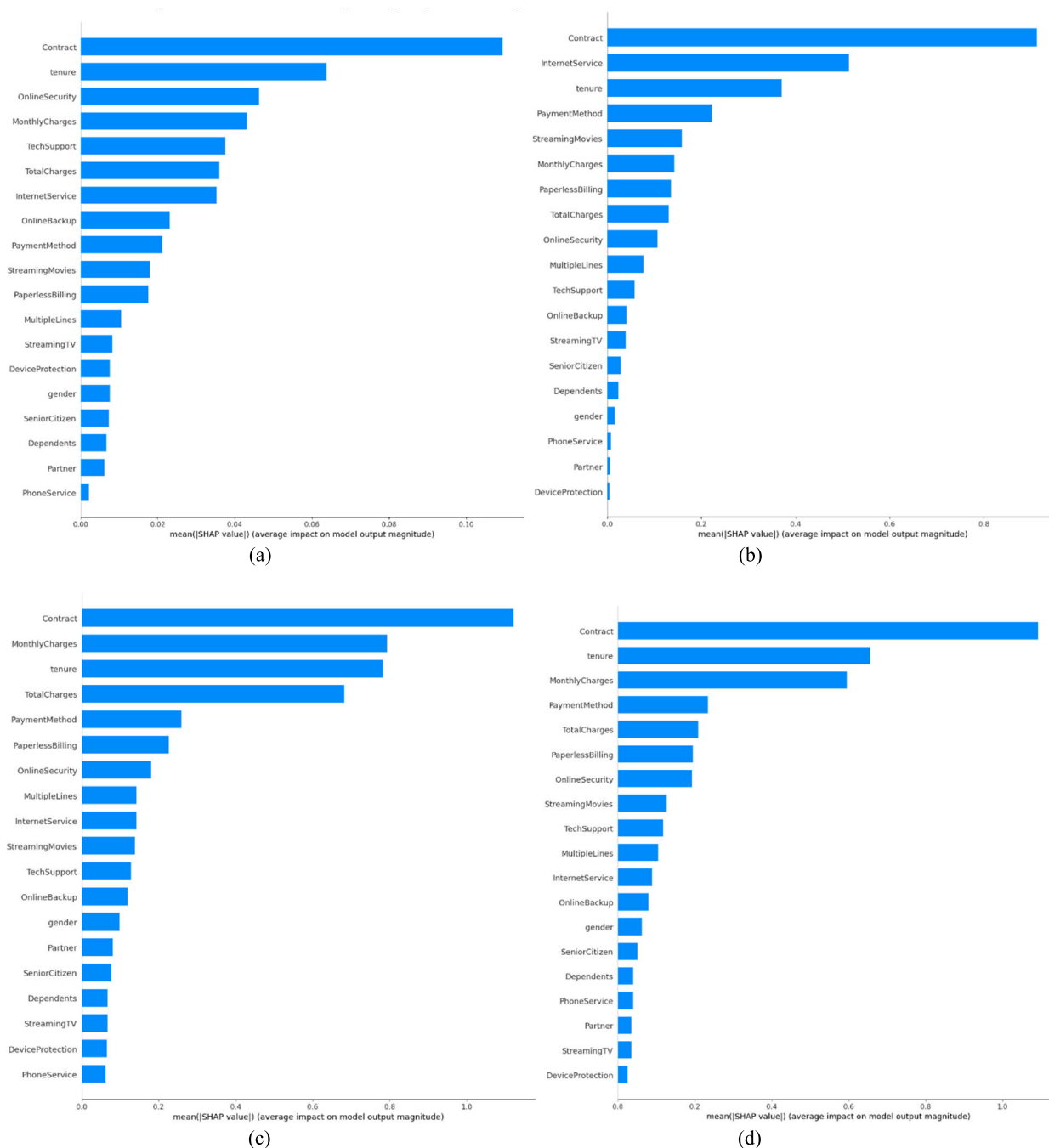


FIGURE 6. SHAP value. Average impact on model output magnitude:(a) BRF, (b) CB, (c) XGB, (d) LGBM.

Tau of 0.7126, which measures concordance while accounting for ties, also demonstrates high agreement in rankings but reflects a slightly weaker relationship than Spearman due to its stricter comparison of rank positions. MAE of 0.0154 shows that the absolute differences between feature importance scores from SHAP and PBE are quite small on average. It suggests a strong alignment in terms of magnitude, meaning that both methods assign similar levels of importance to the features overall. RMSE of 0.0006 reinforces this

observation by penalizing larger differences more heavily. The small RMSE value indicates that discrepancies in the importance scores are minimal and consistent across the features, without any significant outliers. Similar results are obtained for CB, XGB and LGBM.

In ranking analysis, Spearman coefficients above 0.80 are generally considered strong, while values between 0.60 and 0.80 indicate substantial agreement. Similarly, Kendall's Tau values above 0.65 reflect strong pairwise concordance. For

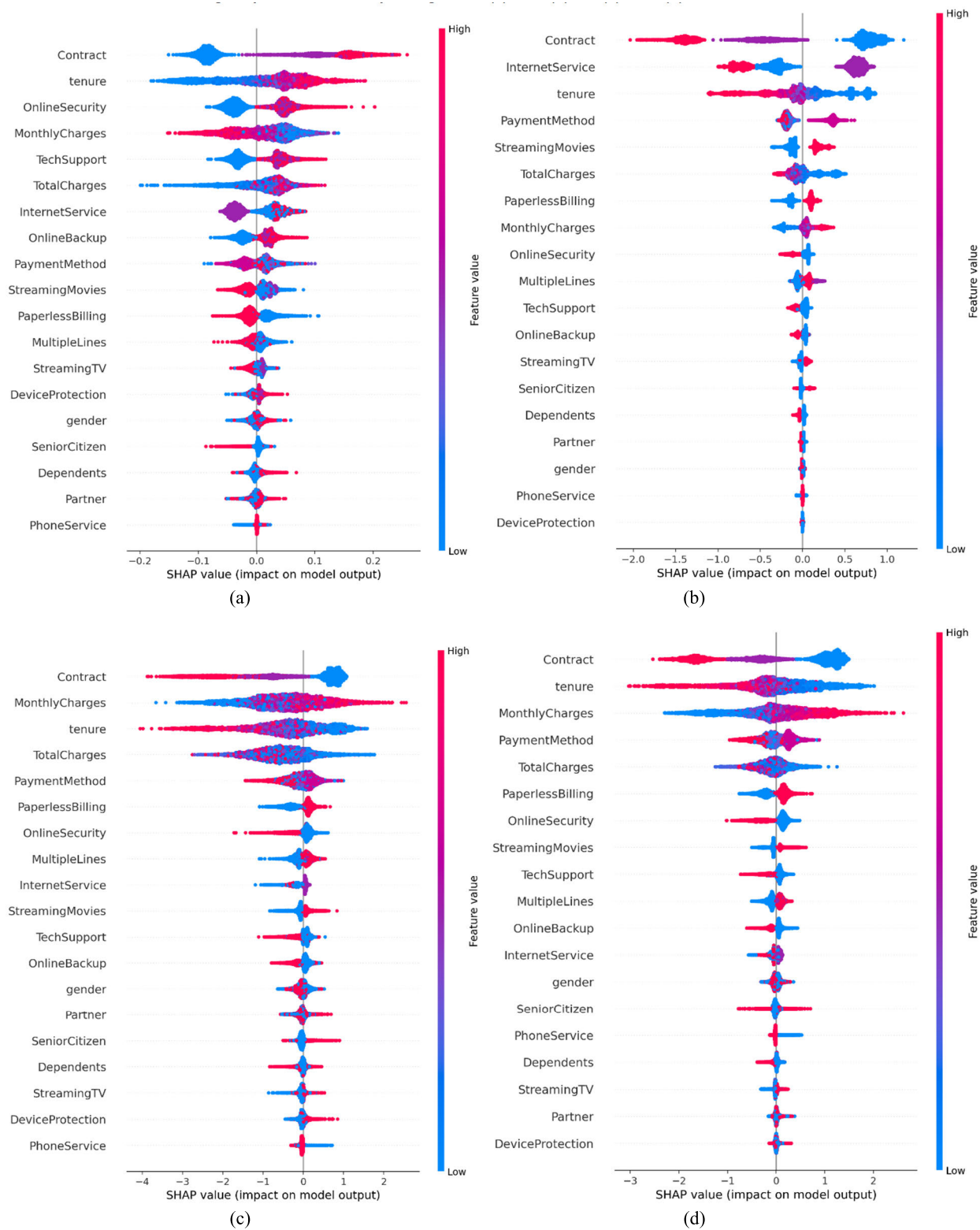


FIGURE 7. SHAP value. Impact on model output for: (a) BRF, (b) CB, (c) XGB, (d) LGBM.

Top-K comparisons, an overlap exceeding 60% (e.g., 3 out of 5 or more) is typically regarded as strong actionable alignment in practical decision-making contexts. Based on these

reference thresholds, the observed correlations and overlap values between SHAP and PBE indicate strong to substantial agreement, supporting the consistency of the proposed frame-

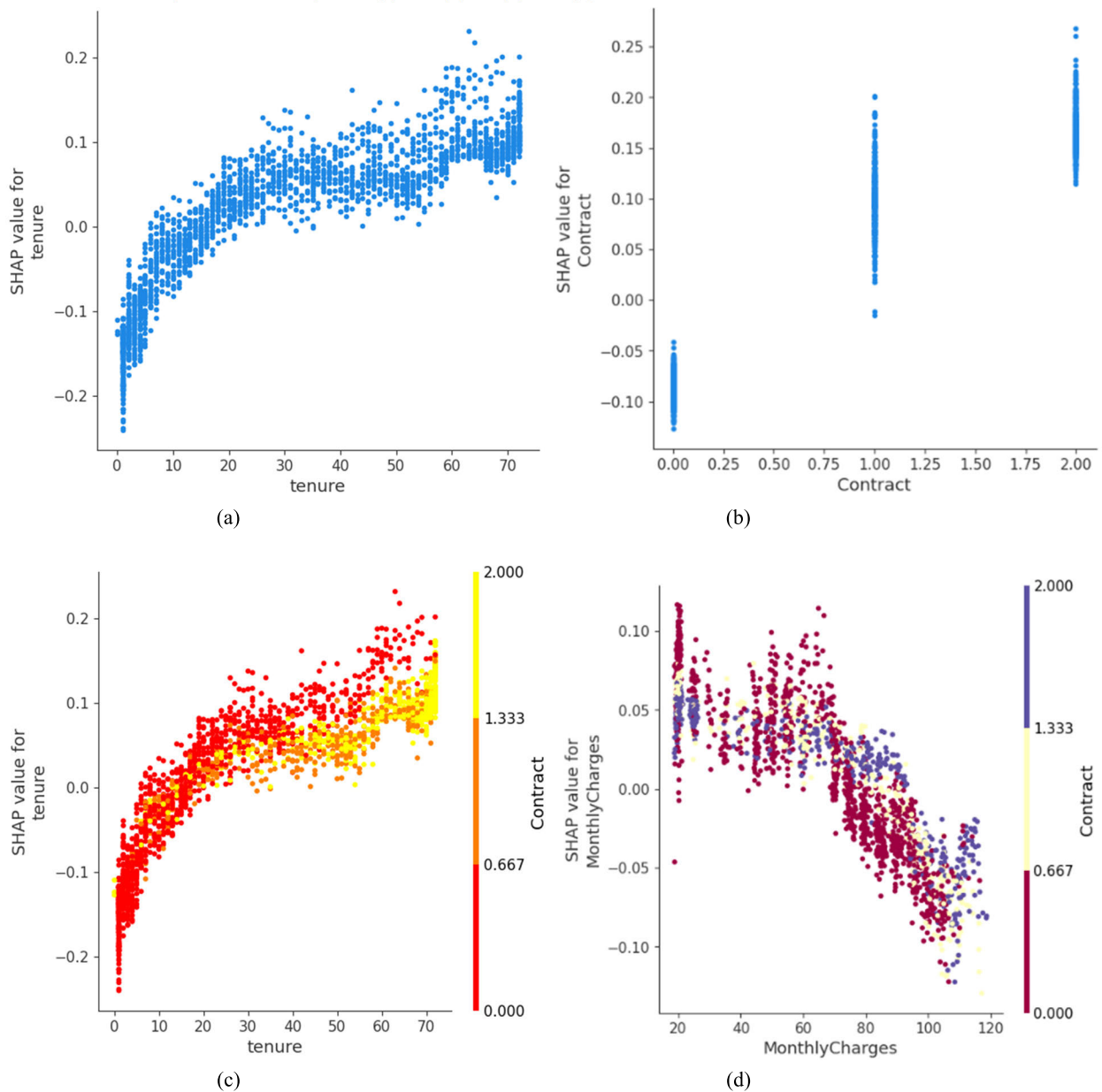


FIGURE 8. BRF + SHAP value for (a) tenure, (b) contract, (c)tenure-contract, (d) monthly charges-contract.

work. The Top-5 Feature Overlap of 3 shows that three out of the top five features identified as most important by SHAP are also identified by the multiple PBE (as in Figure 10). This highlights partial agreement between the methods in identifying the most critical features. Although the Top-5 overlap is 3, the high Spearman (0.8125) and Kendall (0.7126) values indicate strong global ranking consistency. The partial Top-5 mismatch arises from small differences in importance magnitudes among closely ranked features, leading to minor positional swaps rather than structural disagreement. Importantly, the overlapping features correspond to the core churn drivers (contract type, tenure, monthly charges), meaning that the most actionable insights remain consistent across

methods. Therefore, while exact Top-K identity is not perfect, the strategic recommendations derived from SHAP and PBE are aligned. The Top-K is also influenced by the perturbation magnitudes used in RNP and AP. Since feature rankings are derived from recall drops, small changes in *noise_level* or *perturbation_factor* increase Top-K to 4 (for *noise_level* 0.12 and *perturbation_factor* 0.016).

The Jaccard Similarity of 0.6885 indicates that 68.85% of the top-5 features are shared between the two methods, reflecting high agreement in feature selection for the top-ranked variables. Overall, SHAP and PBE show high agreement in feature rankings, strong alignment in importance magnitudes and partial overlap in identifying the most

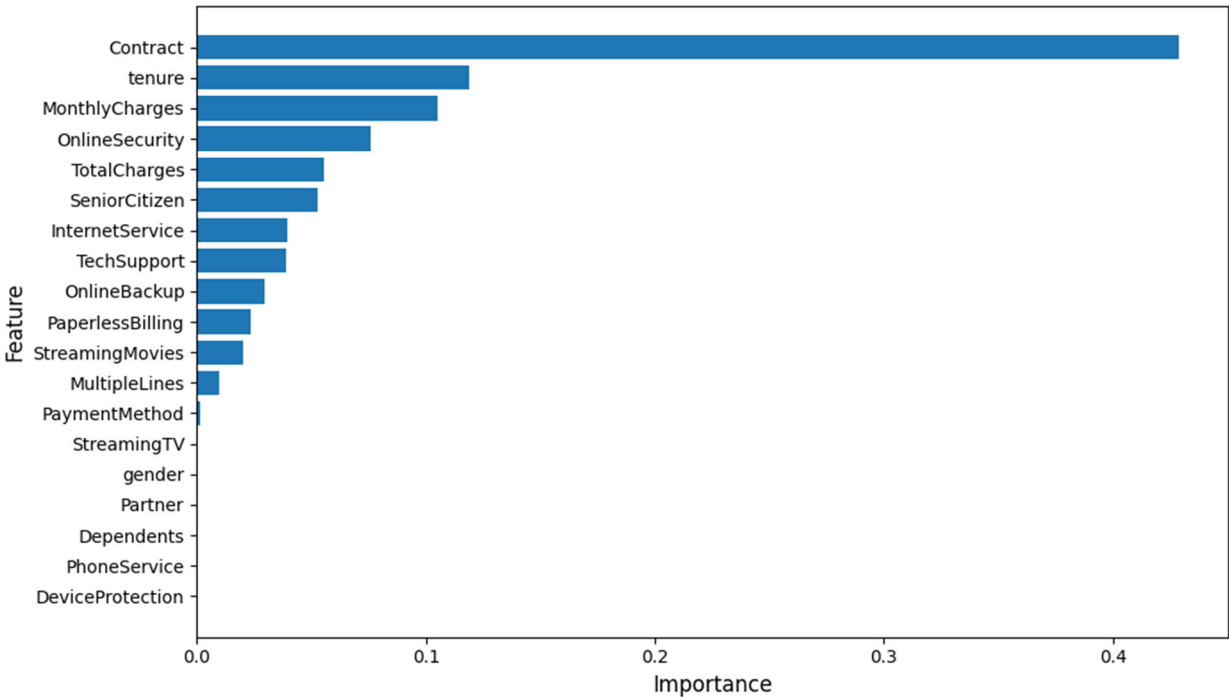


FIGURE 9. LIME features' importance.

TABLE 3. Comparison between features' importance vectors.

Model	Spearman Rank Correlation	Kendall Tau	MAE	RMSE	Top-5 Feature Overlap	Jaccard Similarity	KS Test	EMD
BRF SHAP	0.8125	0.7126	0.0154	0.0006	3	0.6885	0.2105	0.0087
BRF PBE								
CB SHAP	0.7928	0.6792	0.0321	0.0009	3	0.4289	0.2582	0.0091
CB PBE								
XGB SHAP	0.7792	0.6684	0.0256	0.0011	3	0.4420	0.2901	0.0101
XGB PBE								
LGBM SHAP	0.7864	0.6529	0.0359	0.0008	3	0.5019	0.3194	0.0097
LGBM PBE								

important features. It suggests that while the two methods have complementary strengths, some differences in feature rankings and key feature selection likely stem from their inherent methodological differences. SHAP excels in capturing global and local feature importance comprehensively, while multiple PBE provide more intuitive assessments of global feature contributions by measuring performance drops under perturbations.

The Kolmogorov-Smirnov (KS) test is used to determine whether two samples are drawn from the same distribution. KS statistic (0.2105) indicates the maximum absolute difference between the cumulative distribution functions (CDFs) of the two samples, Importance_PBE and Importance_SHAP. A larger value would indicate a greater difference between the two distributions. P-value (0.8081) indicates the probability of observing such a difference (or more extreme)

between the two distributions if they were truly from the same distribution. The Earth Mover's Distance (EMD) is a metric used to quantify the distance between two probability distributions, interpreted as the "minimum amount of work" required to transform one distribution into another. EMD (0.0087) indicates that the two distributions are very similar, as less "work" is needed to make one distribution resemble the other.

KS test and p-value indicate a high probability that the two samples are drawn from the same distribution, supporting the null hypothesis. KS statistic 0.2105 is moderate but does not suggest significant divergence. The very small EMD value (approx. 0.0087) further confirms that the two distributions are nearly identical or only marginally different. The KS test and EMD collectively indicate that the feature importances derived from the multiple PBE and SHAP are highly similar.

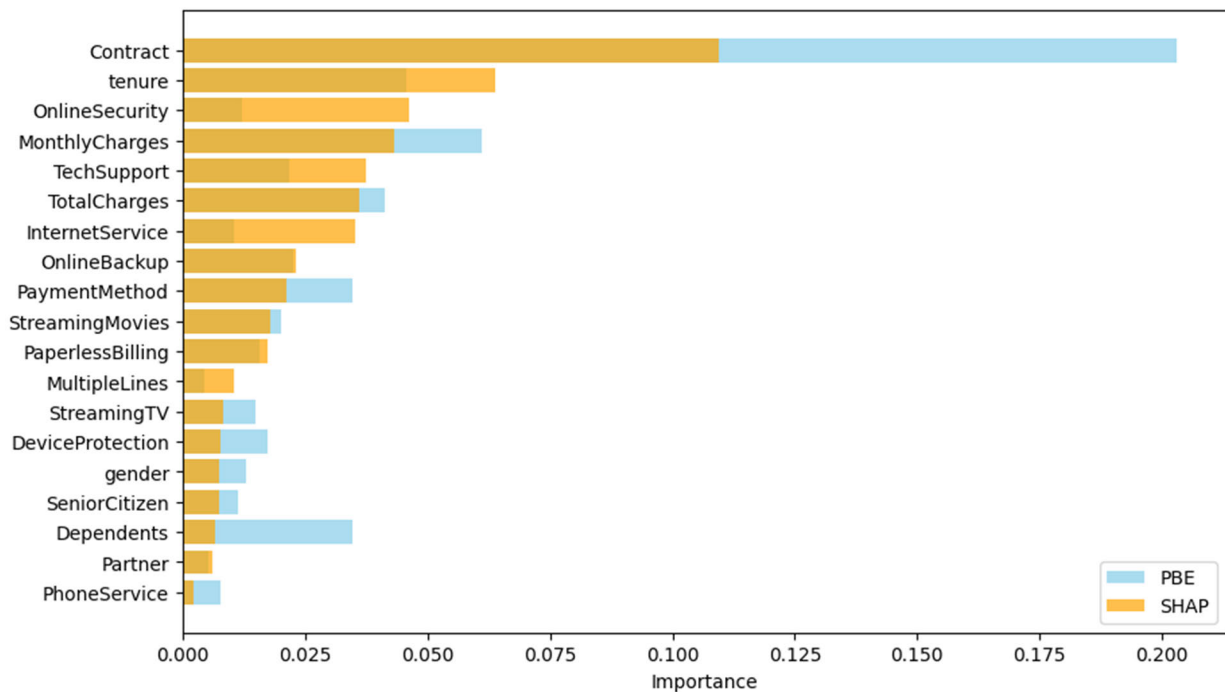


FIGURE 10. Comparison of feature importance: SHAP vs. multiple PBE.

TABLE 4. Rank stability between recall-based and F1-Based feature importance rankings.

Model	Spearman ρ (Recall vs F1)	Kendall τ (Recall vs F1)	Top-5 overlap	Jaccard Similarity
BRF	0.78	0.73	3	0.67
CB	0.76	0.69	3	0.55
XGB	0.75	0.66	3	0.55
LGBM	0.77	0.70	3	0.67

TABLE 5. Comparing SHAP, LIME and multiple PBE.

XAI	Fidelity	Consistency	Runtime
SHAP	0.99	0.64	289.4036
LIME	0.85	0.50	35.0921
PBE	0.84	0.58	18.5172

To assess the robustness of the proposed perturbation-based framework with respect to the evaluation objective, we conduct a sensitivity analysis by recomputing feature importance rankings using F1-score drop instead of recall drop. F1-score balances precision and recall, incorporating both missed churners and false alarms. For each feature, the importance is recalculated as the decrease in F1-score under perturbation and the resulting rankings are compared with the recall-based rankings. Rank stability is evaluated using Spearman's rank correlation and Kendall's Tau, while actionable agreement is assessed through Top-K overlap and Jaccard similarity (as in Table 4).

The results indicate strong agreement between recall-based and F1-based feature importance rankings across all models. Spearman coefficients between 0.75 and 0.78 and Kendall values between 0.66 and 0.73 confirm substantial global rank stability. Although Top-5 overlap equals three features, Jaccard similarity values between 0.55 and 0.67 demonstrate moderate-to-strong actionable agreement.

Table 5 evaluates SHAP, LIME and PBE based on three criteria: fidelity, consistency and runtime. Fidelity measures how well the explanation aligns with the model's predictions, with higher values indicating better alignment. SHAP achieves the highest fidelity at 0.99, meaning it provides highly accurate explanations that closely align with the model's behavior. LIME has moderate fidelity at 0.85, showing reasonable accuracy but not as reliable as SHAP. PBE has the lowest fidelity at 0.84, but just slightly below LIME. Consistency reflects whether the explanation method produces stable and reproducible results when applied multiple times or to similar inputs. PBE is the second ranked after SHAP. Runtime measures the computational efficiency of the method. SHAP has the longest runtime at 289.4036 seconds, making it computationally expensive and impractical for real-time or large-scale applications despite its high fidelity and consistency. LIME has a much faster runtime at 35.0921 seconds, making it significantly more efficient than SHAP and suitable for faster interpretations. PBE is the fastest method with a runtime of 18.5172 seconds, ideal for real-time or resource-constrained environments. It is the best approach for scenarios where speed is the top priority with reasonable appropriate accuracy to SHAP and LIME. The runtime reported in Table 5 corresponds to the SHAP

TABLE 6. Estimating SHAP runtime when handling different datasets.

Dataset Size	Features	Tree SHAP Runtime (Linear)	Kernel SHAP Runtime (Quadratic)
7,000 instances	21	300 seconds (~5 min)	300 seconds (~5 min)
10,000 instances	21	~428 seconds (~7.1 min)	~612 seconds (~10.2 min)
50,000 instances	21	~2,143 seconds (~35.7 min)	~15,306 seconds (~4.25 hours)
100,000 instances	21	~4,286 seconds (~71.4 min)	~61,224 seconds (~17 hours)
500,000 instances	21	~21,430 seconds (~6 hours)	~1,530,612 seconds (~425 hours)
1,000,000 instances	21	~42,860 seconds (~12 hours)	~6,122,449 seconds (~71 days)
10,000 instances	50	~1,020 seconds (~17 min)	~2,040 seconds (~34 min)
50,000 instances	50	~5,100 seconds (~85 min)	~51,020 seconds (~14 hours)
100,000 instances	50	~10,200 seconds (~2.8 hours)	~204,082 seconds (~56 hours)
1,000,000 instances	50	~102,000 seconds (~28 hours)	~20,408,163 seconds (~236 day)

TreeExplainer implementation, which is specifically optimized for tree-based models.

SHAP's runtime typically scales linearly or quadratically with the number of instances, depending on the algorithm used (e.g., kernel SHAP or tree SHAP) and the model type (e.g., deep learning models vs. tree-based models). Table 6 estimates SHAP runtime for different dataset sizes.

FA represents the most computationally demanding component of the proposed PBE framework, as it requires model retraining after the removal of each feature, resulting in complexity proportional to the number of features. While this is manageable in moderate-dimensional settings (e.g., 50–100 features), scalability may become challenging in high-dimensional datasets. However, the PBE framework is modular, and FA can be selectively applied or approximated. Several acceleration strategies can be employed, including parallel retraining, warm-start mechanisms for tree-based models, early stopping criteria and screening approaches where FA is applied only to a subset of candidate features identified through faster perturbation methods (e.g., RNP or IM).

The selected perturbation parameters were not intended as optimal tuning values but as moderate perturbation magnitudes designed to introduce controlled variability without distorting the data distribution. To ensure robustness, we conducted a sensitivity analysis by varying noise_level within [0.05, 0.2] and perturbation_factor within [0.005, 0.02]. The resulting feature rankings remained highly stable, with Spearman correlations above 0.81 across configurations and consistent Top-K overlap. These results confirm that the unified PBE ranking is robust to reasonable perturbation magnitudes and not driven by a specific parameter choice.

V. CONCLUSION

This research proposed a comprehensive methodology for explaining AI models and identifying the most critical features for retaining customers to prevent churn. By aggregating feature importance values across several explainers (RNP, FA, IM and AP), the multiple PBE algorithm integrates diverse perturbations into a unified importance score, reducing bias and variability associated with individual techniques.

The methodology highlights impactful features while comparing its results against established techniques like SHAP and LIME.

The evaluation of telecommunication customer data revealed that recall is the most critical metric in churn prediction, as it measures the ability to identify potential churners effectively. Among the classifiers analyzed, Balanced Random Forest (BRF) provided the best results in terms of overall performance, outperforming other models like CatBoost, XGBoost and LightGBM. Features driving churn include contract type, tenure and monthly charges. Customers on month-to-month contracts are at the highest risk of churn, especially those with a tenure below 20 months. Transitioning these customers to longer contracts, such as one-year or two-year agreements, may significantly reduce churn. Customers with monthly charges exceeding \$90 are also highly likely to churn. To retain such customers, stakeholders should reassess pricing strategies, introduce value-added services and offer personalized discounts.

New customers with a tenure of less than 20 months are particularly vulnerable to churn. Early engagement programs, such as personalized onboarding, loyalty rewards and proactive customer support, are critical during this period to build loyalty and trust. For customers with mid-range monthly charges (\$60–\$90), churn risk is influenced by additional factors such as contract type and tenure.

The methodology also enables businesses to implement real-time retention strategies by identifying high-risk customers dynamically. Automated interventions, including dynamic pricing adjustments, targeted promotions and personalized retention efforts, help businesses mitigate churn proactively. Moreover, the analysis highlights that customers value affordability and stability. Stakeholders should emphasize premium customer support and loyalty incentives to improve long-term retention.

Comparative analyses showed strong alignment between SHAP and the proposed PBE methodology in terms of feature importance rankings and magnitudes. Statistical tests, including Spearman Rank Correlation, Kendall Tau and Jaccard Similarity, demonstrated high agreement between the methods. Kolmogorov-Smirnov and Earth Mover's Distance tests

further confirmed the similarity between feature importance distributions derived from SHAP and PBE.

SHAP, LIME and PBE highlighted trade-offs between fidelity, consistency and runtime. SHAP achieved the highest fidelity but suffered from computational inefficiencies, particularly for larger datasets. LIME offered faster runtimes but with moderate fidelity and lower consistency. PBE emerged as the most efficient method, achieving reasonable fidelity and consistency, making it particularly suitable for real-time and large-scale applications. Runtime scalability analyses further emphasized the feasibility of PBE for handling big data scenarios compared to SHAP. Future research could extend the proposed methodology to other domains and further optimize computational performance for high-dimensional datasets.

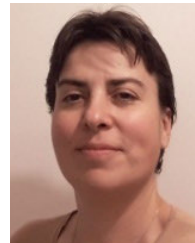
From a practical deployment perspective, the proposed framework can be integrated into Customer Relationship Management (CRM) systems as a decision-support layer accompanying churn prediction models. Given its moderate runtime, PBE is well suited for batch-based monitoring (e.g., daily or weekly updates), where explanations for high-risk customers are generated and visualized through API-driven dashboards. In dynamic environments, data and concept drift may alter feature–churn relationships over time. Periodic recalculation of feature importance rankings, combined with rank stability monitoring, can serve as a drift detection mechanism, triggering model retraining when substantial deviations occur.

Although the proposed multiple PBE framework is structurally model-agnostic and perturbation-based, certain aspects of the conclusions remain domain-dependent. In this study, recall was selected as the primary optimization objective due to the high cost of false negatives in churn prediction; however, alternative domains may prioritize different utility functions (e.g., precision, F1-score, AUC), which may slightly alter feature importance rankings. Additionally, the present evaluation was conducted on a structured, moderate-dimensional tabular dataset. In high-dimensional, sparse or highly correlated feature spaces, perturbation responses may exhibit different stability characteristics and approximate or staged FA strategies may be required for scalability. Thus, while the methodological framework generalizes across domains, the interpretation of “most important” features should be understood as conditional on the selected performance objective and data structure.

REFERENCES

- [1] V. Pasquadibisceglie, A. Appice, G. Ieva, and D. Malerba, “TSUNAMI—An explainable PPM approach for customer churn prediction in evolving retail data environments,” *J. Intell. Inf. Syst.*, vol. 62, no. 3, pp. 705–733, Jun. 2024, doi: [10.1007/s10844-023-00838-5](https://doi.org/10.1007/s10844-023-00838-5).
- [2] D. Slack, S. Hilgard, S. Singh, and H. Lakkaraju, “Reliable post hoc explanations: Modeling uncertainty in explainability,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 1–14.
- [3] X. Man and E. P. Chan, “The best way to select features? Comparing MDA, LIME, and SHAP,” *J. Financial Data Sci.*, vol. 3, no. 1, pp. 127–139, Jan. 2021, doi: [10.3905/jfds.2020.1.047](https://doi.org/10.3905/jfds.2020.1.047).
- [4] M. Veerappa, M. Anneken, N. Burkart, and M. F. Huber, “Validation of XAI explanations for multivariate time series classification in the maritime domain,” *J. Comput. Sci.*, vol. 58, Feb. 2022, Art. no. 101539, doi: [10.1016/j.jocs.2021.101539](https://doi.org/10.1016/j.jocs.2021.101539).
- [5] D. Slack, S. Hilgard, E. Jia, S. Singh, and H. Lakkaraju, *Fooling LIME and SHAP*. New York, NY, USA: Association for Computing Machinery, 2020, doi: [10.1145/3375627.3375830](https://doi.org/10.1145/3375627.3375830).
- [6] I. Gómez-Talal, L. Bote-Curiel, and J. L. Rojo-Álvarez, “Understanding the disparities in mathematics performance: An interpretability-based examination,” *Eng. Appl. Artif. Intell.*, vol. 133, Jul. 2024, Art. no. 108109, doi: [10.1016/j.engappai.2024.108109](https://doi.org/10.1016/j.engappai.2024.108109).
- [7] W. Jin, X. Li, M. Fatehi, and G. Hamarneh, “Generating post-hoc explanation from deep neural networks for multi-modal medical image analysis tasks,” *MethodsX*, vol. 10, Jan. 2023, Art. no. 102009, doi: [10.1016/j.mex.2023.102009](https://doi.org/10.1016/j.mex.2023.102009).
- [8] C. Ji, R. Wang, and H. Wu, “Perturb more, trap more: Understanding behaviors of graph neural networks,” *Neurocomputing*, vol. 493, pp. 59–75, Jul. 2022, doi: [10.1016/j.neucom.2022.04.070](https://doi.org/10.1016/j.neucom.2022.04.070).
- [9] E. P. Xing, Q. Ho, P. Xie, and D. Wei, “Strategies and principles of distributed machine learning on big data,” *Engineering*, vol. 2, no. 2, pp. 179–195, Jun. 2016, doi: [10.1016/j.eng.2016.02.008](https://doi.org/10.1016/j.eng.2016.02.008).
- [10] D. Gaspar, P. Silva, and C. Silva, “Explainable AI for intrusion detection systems: LIME and SHAP applicability on multi-layer perceptron,” *IEEE Access*, vol. 12, pp. 30164–30175, 2024, doi: [10.1109/ACCESS.2024.3368377](https://doi.org/10.1109/ACCESS.2024.3368377).
- [11] C. P. Vieira and L. A. Digiampietri, “Machine learning post-hoc interpretability: A systematic mapping study,” in *Proc. XVIII Brazilian Symp. Inf. Syst.*, May 2022, pp. 1–8, doi: [10.1145/3535511.3535512](https://doi.org/10.1145/3535511.3535512).
- [12] J. Zhou and F. Chen, “Visible, explainable, trustworthy and transparent,” in *Human and Machine Learning* (Human–Computer Interaction Series). Cham, Switzerland: Springer, Jun. 2018. [Online]. Available: <https://doi.org/10.1007/978-3-319-90403-0>
- [13] M. N. Vu and M. T. Thai, “Limitations of perturbation-based explanation methods for temporal graph neural networks,” in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Dec. 2023, pp. 618–627, doi: [10.1109/ICDM58522.2023.00071](https://doi.org/10.1109/ICDM58522.2023.00071).
- [14] N. Jethani, M. Sudarshan, I. Covert, S. I. Lee, and R. Ranganath, “FastSHAP: Real-time Shapley value estimation,” in *Proc. 10th Int. Conf. Learn. Represent. (ICLR)*, 2022, pp. 1–23. [Online]. Available: <https://par.nsf.gov/servlets/purl/10350825>
- [15] S. Bengamra, O. Mzoughi, A. Bigand, and E. Zagrouba, “Towards explainability in using deep learning for face detection in paintings,” in *Proc. 12th Int. Conf. Pattern Recognit. Appl. Methods*, 2023, pp. 832–841, doi: [10.5220/0011670300003411](https://doi.org/10.5220/0011670300003411).
- [16] C. Jean-Quartier, K. Bein, L. Hejny, E. Hofer, A. Holzinger, and F. Jeanquartier, “The cost of understanding—XAI algorithms towards sustainable ML in the view of computational cost,” *Computation*, vol. 11, no. 5, p. 92, May 2023, doi: [10.3390/computation11050092](https://doi.org/10.3390/computation11050092).
- [17] D. Y. C. Wang, L. A. Jordanger, and J. C.-W. Lin, “Explainability of leverage points exploration for customer churn prediction,” in *Proc. IEEE Int. Conf. Big Data (BigData)*, Jordan, Dec. 2023, pp. 5997–6004, doi: [10.1109/BigData59044.2023.10386857](https://doi.org/10.1109/BigData59044.2023.10386857).
- [18] S. T. Eswarapu, S. Seshathri, Y. Deshaboina, P. Bhargawa, A. A. Jo, and E. D. Raj, “Integrated customer analytics using explainability and AutoML for telecommunications,” in *Proc. 2nd Int. Conf. Appl. Artif. Intell. Comput. (ICAAIC)*, May 2023, pp. 228–235, doi: [10.1109/ICAAIC56838.2023.10141019](https://doi.org/10.1109/ICAAIC56838.2023.10141019).
- [19] R. Lukyanenko, A. Castellanos, V. C. Storey, A. Castillo, M. C. Tremblay, and J. Parsons, “Superimposition: Augmenting machine learning outputs with conceptual models for explainable AI,” in *Advances in Conceptual Modeling* (Lecture Notes in Computer Science), vol. 12584, G. Grossmann and S. Ram, Eds., Cham, Switzerland: Springer, Dec. 2020, doi: [10.1007/978-3-030-65847-2_3](https://doi.org/10.1007/978-3-030-65847-2_3).
- [20] H. M. Ngie, L. Nderu, L. Mutanu, and D. M. Gicuku, “Mitigating pre-conception in machine learning classifiers,” in *Proc. Int. Conf. Electr., Comput., Commun. Mechatronics Eng. (ICECCME)*, Oct. 2021, pp. 1–6, doi: [10.1109/ICECCME52200.2021.9591043](https://doi.org/10.1109/ICECCME52200.2021.9591043).
- [21] C. K. Leung, A. G. M. Pazdor, and J. Souza, “Explainable artificial intelligence for data science on customer churn,” in *Proc. IEEE 8th Int. Conf. Data Sci. Adv. Anal. (DSAA)*, Oct. 2021, pp. 1–10, doi: [10.1109/DSAA53316.2021.9564166](https://doi.org/10.1109/DSAA53316.2021.9564166).
- [22] D. Minh, H. X. Wang, Y. F. Li, and T. N. Nguyen, “Explainable artificial intelligence: A comprehensive review,” *Artif. Intell. Rev.*, vol. 55, no. 5, pp. 3503–3568, Jun. 2022, doi: [10.1007/s10462-021-10088-y](https://doi.org/10.1007/s10462-021-10088-y).

- [23] A. Messalas, Y. Kanellopoulos, and C. Makris, "Model-agnostic interpretability with Shapley values," in *Proc. 10th Int. Conf. Inf., Intell., Syst. Appl. (IISA)*, Jul. 2019, pp. 1–7, doi: [10.1109/IISA.2019.8900669](https://doi.org/10.1109/IISA.2019.8900669).
- [24] J. Tao, Y. Xiong, S. Zhao, R. Wu, X. Shen, T. Lyu, C. Fan, Z. Hu, S. Zhao, and G. Pan, "Explainable AI for cheating detection and churn prediction in online games," *IEEE Trans. Games*, vol. 15, no. 2, pp. 242–251, Jun. 2023, doi: [10.1109/TG.2022.3173399](https://doi.org/10.1109/TG.2022.3173399).
- [25] I. Weiss and D. Vilenchik, "Predicting churn in online games by quantifying diversity of engagement," *Big Data*, vol. 11, no. 4, pp. 282–295, Aug. 2023, doi: [10.1089/big.2022.0109](https://doi.org/10.1089/big.2022.0109).
- [26] Y. Xiong, R. Wu, S. Zhao, J. Tao, X. Shen, T. Lyu, C. Fan, and P. Cui, "A data-driven decision support framework for player churn analysis in online games," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2023, pp. 5303–5314, doi: [10.1145/3580305.3599759](https://doi.org/10.1145/3580305.3599759).
- [27] J. Tao, Y. Xiong, S. Zhao, Y. Xu, J. Lin, R. Wu, and C. Fan, "XAI-driven explainable multi-view game cheating detection," in *Proc. IEEE Conf. Games (CoG)*, Aug. 2020, pp. 144–151, doi: [10.1109/CoG47356.2020.9231843](https://doi.org/10.1109/CoG47356.2020.9231843).
- [28] J. Mohajon and A. S. Mohammad Arif, "Churn prediction with explainability for the customers of telecom industry," in *Proc. Int. Conf. Inf. Commun. Technol. Sustain. Develop. (ICICT4SD)*, Sep. 2023, pp. 179–183, doi: [10.1109/ICICT4SD5951.2023.10303395](https://doi.org/10.1109/ICICT4SD5951.2023.10303395).
- [29] I. Ullah, A. Rios, V. Gala, and S. McKeever, "Explaining deep learning models for tabular data using layer-wise relevance propagation," *Appl. Sci.*, vol. 12, no. 1, p. 136, Dec. 2021, doi: [10.3390/app12010136](https://doi.org/10.3390/app12010136).
- [30] M. Szelag and R. Słowiński, "Explaining and predicting customer churn by monotonic rules induced from ordinal data," *Eur. J. Oper. Res.*, vol. 317, no. 2, pp. 414–424, Sep. 2024, doi: [10.1016/j.ejor.2023.09.028](https://doi.org/10.1016/j.ejor.2023.09.028).
- [31] S. Murindanyi, B. W. Mugalu, J. Nakatumba-Nabende, and G. Marvin, "Interpretable machine learning for predicting customer churn in retail banking," in *Proc. 7th Int. Conf. Trends Electron. Informat. (ICOEI)*, Apr. 2023, pp. 967–974, doi: [10.1109/ICOEI56765.2023.10125859](https://doi.org/10.1109/ICOEI56765.2023.10125859).
- [32] A.-G. Văduva, S.-V. Oprea, A.-M. Niculae, A. Băra, and A.-I. Andreescu, "Improving churn detection in the banking sector: A machine learning approach with probability calibration techniques," *Electronics*, vol. 13, no. 22, p. 4527, Nov. 2024, doi: [10.3390/electronics13224527](https://doi.org/10.3390/electronics13224527).
- [33] G. Marín Díaz, J. J. Galán, and R. A. Carrasco, "XAI for churn prediction in B2B models: A use case in an enterprise software company," *Mathematics*, vol. 10, no. 20, p. 3896, Oct. 2022, doi: [10.3390/math10203896](https://doi.org/10.3390/math10203896).
- [34] G. Andresini et al., "CENTAURO: An explainable AI approach for customer loyalty prediction in retail sector," in *Advances in Artificial Intelligence* (Lecture Notes in Computer Science), vol. 14318, R. Basili, D. Lembo, C. Limongelli, and A. Orlandini, Eds., Cham, Switzerland: Springer, Nov. 2023, doi: [10.1007/978-3-031-47546-7_14](https://doi.org/10.1007/978-3-031-47546-7_14).
- [35] A. Gramegna and P. Giudici, "Why to buy insurance? An explainable artificial intelligence approach," *Risks*, vol. 8, no. 4, p. 137, Dec. 2020, doi: [10.3390/risks8040137](https://doi.org/10.3390/risks8040137).
- [36] M. Katsuragi and K. Tanaka, "Dropout prediction by interpretable machine learning model towards preventing student dropout," in *Transdisciplinarity and the Future of Engineering* (Advances in Transdisciplinary Engineering), vol. 28, 2022, pp. 678–683, doi: [10.3233/ATDE220700](https://doi.org/10.3233/ATDE220700).
- [37] Z. Mingyu, W. Sutong, W. Yanzhang, and W. Dujuan, "An interpretable prediction method for university student academic crisis warning," *Complex Intell. Syst.*, vol. 8, no. 1, pp. 323–336, Feb. 2022, doi: [10.1007/s40747-021-00383-0](https://doi.org/10.1007/s40747-021-00383-0).
- [38] S. M. Lundberg and S. I. Lee, "A unified approach to interpreting model predictions," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1–10.
- [39] R. Wang, X. Wang, and D. I. Inouye, "Shapley explanation networks," in *Proc. 9th Int. Conf. Learn. Represent. (ICLR)*, 2021, pp. 1–26.
- [40] I. C. Covert, S. Lundberg, and S. I. Lee, "Understanding global feature contributions with additive importance measures," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1–12.
- [41] I. C. Covert, S. Lundberg, and S. I. Lee, "Explaining by removing: A unified framework for model explanation," *J. Mach. Learn. Res.*, vol. 22, pp. 1–90, Jan. 2021.
- [42] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Mach. Intell.*, vol. 2, no. 1, pp. 56–67, Jan. 2020, doi: [10.1038/s42256-019-0138-9](https://doi.org/10.1038/s42256-019-0138-9).
- [43] S. Knapič, A. Malhi, R. Saluja, and K. Främling, "Explainable artificial intelligence for human decision support system in the medical domain," *Mach. Learn. Knowl. Extraction*, vol. 3, no. 3, pp. 740–770, Sep. 2021, doi: [10.3390/make3030037](https://doi.org/10.3390/make3030037).
- [44] M. Karim Belaid, E. Hüllermeier, M. Rabus, and R. Krestel, "Do we need another explainable AI method? Toward unifying post-hoc XAI evaluation methods into an interactive and multi-dimensional benchmark," 2022, *arXiv:2207.14160*.
- [45] K. Kaczmarek-Majer, G. Casalino, G. Castellano, M. Dominiak, O. Hryniewicz, O. Kamińska, G. Vessio, and N. Díaz-Rodríguez, "PLE-NARY: Explaining black-box models in natural language through fuzzy linguistic summaries," *Inf. Sci.*, vol. 614, pp. 374–399, Oct. 2022, doi: [10.1016/j.ins.2022.10.010](https://doi.org/10.1016/j.ins.2022.10.010).
- [46] G. Babaei and P. Giudici, "InstanceSHAP: An instance-based estimation approach for Shapley values," *Behaviormetrika*, vol. 51, no. 1, pp. 425–439, Jan. 2024, doi: [10.1007/s41237-023-00208-z](https://doi.org/10.1007/s41237-023-00208-z).
- [47] H. Meng, C. Wagner, and I. Triguero, "Explaining time series classifiers through meaningful perturbation and optimisation," *Inf. Sci.*, vol. 645, Oct. 2023, Art. no. 119334, doi: [10.1016/j.ins.2023.119334](https://doi.org/10.1016/j.ins.2023.119334).
- [48] C. Agarwal and A. Nguyen, "Explaining image classifiers by removing input features using generative models," in *Computer Vision—ACCV 2020* (Lecture Notes in Computer Science), H. Ishikawa, C. L. Liu, T. Pajdla, and J. Shi, vol. 12627, Cham, Switzerland: Springer, Feb. 2021, doi: [10.1007/978-3-030-69544-6_7](https://doi.org/10.1007/978-3-030-69544-6_7).



SIMONA-VASILICA OPREA was born in July 1978. She received the M.Sc. degree from the Infrastructure Management Program, Yokohama National University, Japan, in 2007, the first Ph.D. degree in power system engineering from Bucharest Polytechnic University, in 2009, and the second Ph.D. degree in economic informatics from Bucharest University of Economic Studies, in 2017. She currently teaches databases, database management systems, and software packages with the Faculty of Economic Cybernetics, Statistics and Informatics, Bucharest University of Economic Studies.



ADELA BĂRA was born in October 1978. She is currently the Director of the Data Science Excellence Center, Bucharest University of Economic Studies. She is the author of 19 books in the domain of economic informatics, over 60 published scientific papers and articles. She has participated in ten research projects, financed from national and international research programs. Her research interests include databases, data warehousing, big data, data mining, artificial neural networks, machine learning, the IoT, business intelligence, and informatics solutions for energy systems.

...